

Deep Learning for Artificial Intelligence Final Project Report

Time-Series Data and Application to Stock Markets

Student ID: 230051

Course: Deep Learning for Artificial Intelligence (DL4AI)

Date: May 2026



Contents

Abstract	3
1 Introduction	3
1.1 Objectives	3
1.2 Research Questions	4
2 Datasets	4
2.1 Nasdaq Dataset	4
2.2 Vietnam Stock Dataset (VNINDEX)	5
2.3 Technical Indicators (12 features added to Vietnam data)	5
2.4 Data Split Strategy	5
3 Methodology	6
3.1 Sliding Window Construction	6
3.2 Normalization	6
3.3 CNN-LSTM Model Architecture	7
3.4 Training Configuration	8
3.5 Trading Signal Definition (Section 3)	8
3.6 Portfolio Management Methodology (Section 4)	8
4 Experimental Results	9
4.1 Section 1 — Nasdaq Price Prediction	9
4.1.1 Task 1.1 — AAPL Multi-feature (next-day Close)	9
4.1.2 Task 1.2 — k -th Day Ahead Forecast	9
4.1.3 Task 1.3 — k Consecutive Days Forecast	10
4.2 Section 2 — Vietnam Price Prediction	10
4.3 Section 3 — Trading Signal Classification	10
4.4 Section 4 — Portfolio Management	11
4.4.1 Task 4.1 — Profitable Company Selection	11

4.4.2	Task 4.2 — Risk Identification	11
4.4.3	Task 4.3 — Portfolio Evaluation	12
4.5	Section 5 — Model Deployment	12
4.5.1	Task 5.1 — REST API (Flask)	12
4.5.2	Task 5.2 — Web-based SaaS Dashboard (Streamlit)	12
4.5.3	Task 5.3 — AI Automation Workflow (Apache Airflow + SQLite)	13
5	Discussion	14
5.1	Strengths	14
5.2	Limitations	15
5.3	Comparison with Literature	15
6	Conclusion	15
	References	16

Abstract

This project applies deep learning to financial time-series data from two markets: Nasdaq (USA) and the Vietnam stock exchange (VNINDEX). A unified **CNN-LSTM** (Convolutional Neural Network — Long Short-Term Memory) architecture is designed and applied across five tasks: (1) multi-feature next-day price prediction on Nasdaq, (2) price prediction with technical indicators on Vietnam data, (3) binary trading signal classification (BUY / SELL), (4) quantitative portfolio construction and evaluation, and (5) production deployment as a full-stack service — including a Flask REST API, a multi-page **Streamlit SaaS dashboard**, and an **Apache Airflow automation pipeline** backed by SQLite. Experimental results show that the model achieves low MAPE ($\approx 1\text{--}2\%$) for short-horizon price regression, moderate signal classification performance (AUC 0.46–0.53), both constructed portfolios outperform the market benchmark by +5.6 percentage points during the 2022 evaluation period, and the daily pipeline reliably generates 19 predictions across 11 tickers in under 60 seconds.

1. Introduction

Stock market forecasting is a long-standing and difficult problem in quantitative finance due to the inherently noisy, non-stationary, and high-dimensional nature of financial time-series data. Classical approaches such as ARIMA, GARCH, and hand-crafted technical analysis rules have been largely superseded by machine learning methods that can learn non-linear patterns from raw data.

Among deep learning models, **Long Short-Term Memory (LSTM)** networks are naturally suited to sequential data but can struggle to efficiently extract local short-term patterns from raw OHLCV features. **Convolutional Neural Networks (CNN)**, while primarily known for image tasks, are effective at extracting local temporal features in one dimension. Combining the two — a **CNN-LSTM** architecture — leverages the strengths of both: CNN layers extract local candlestick-level patterns, and the LSTM layer models long-range temporal dependencies.

1.1. Objectives

This project addresses five interconnected tasks:

1. **Nasdaq price forecasting** — next-day price, k -th day ahead, and k consecutive days multi-output.

2. **Vietnam price forecasting** — incorporating 12 technical indicators alongside OHLCV features.
3. **Trading signal classification** — predicting BUY and SELL signals from historical price features.
4. **Portfolio management** — constructing and back-testing two investor-profile portfolios (risk-taking vs. prudent) from the VNINDEX universe.
5. **Deployment** — REST API (Flask), interactive web SaaS (Streamlit), automated daily pipeline (Airflow + SQLite), and Docker containerisation.

1.2. Research Questions

- Can a CNN-LSTM model accurately predict short-horizon stock prices using only OHLCV features?
- Do technical indicators improve forecasting on Vietnam stock data?
- How reliably can a classifier detect BUY and SELL trading signals from historical price data alone?
- Does a model-driven signal strategy outperform a buy-and-hold baseline?
- How can a trained deep learning model be served as a production-ready REST API?
- Can the inference workflow be automated and operationalised as a daily data pipeline?

2. Datasets

2.1. Nasdaq Dataset

Table 1: Nasdaq dataset properties

Property	Detail
Source	<code>data_nasdaq_csv/csv/</code> — one CSV per ticker
Coverage	~2,500+ US-listed Nasdaq stocks, daily data
Date range	Varies per ticker (AAPL: 1984–2016+)
Features	Low, Open, Volume, High, Close, Adjusted Close (6 columns)
Target	Close price
Demo ticker	AAPL (Apple Inc.) — longest complete history
Preprocessing	Sort by date, drop NaN rows, build 30-day sliding windows

2.2. Vietnam Stock Dataset (VNINDEX)

Table 2: Vietnam dataset properties

Property	Detail
Source	data-vn-20230228/stock-historical-data/
Filename format	{TICKER}-{EXCHANGE}-History.csv
Coverage	416 VNINDEX-listed tickers, data through February 2023
Features	Open, High, Low, Close, Volume (5 base) + 12 technical indicators = 17 features
Target	Close price
Demo ticker	REE (Refrigeration Electrical Engineering Corporation)
Supplementary	Quarterly financial ratios from financial-ratio/ (D/E, ROE, ROA, P/E, P/B)

2.3. Technical Indicators (12 features added to Vietnam data)

Table 3: Technical indicators computed for Vietnam data

Category	Indicator	Description
Trend	SMA-10, SMA-20	10-day and 20-day Simple Moving Average
Momentum	MACD	EMA(12) – EMA(26)
Momentum	MACD Signal	9-day EMA of MACD
Momentum	MACD Histogram	MACD – MACD Signal
Oscillator	RSI-14	Relative Strength Index (14-day)
Volatility	BB Upper / BB Lower	Bollinger Band boundaries (20-day, $\pm 2\sigma$)
Volatility	BB Width	BB Upper – BB Lower
Volatility	BB %B	Position of price within bands
Volatility	ATR-14	Average True Range (14-day)
Volume	OBV	On-Balance Volume (cumulative signed volume)

2.4. Data Split Strategy

All experiments use a **strictly chronological split** to prevent data leakage:

Table 4: Train/validation/test split

Partition	Ratio	Purpose
Train	70 %	Model parameter fitting
Validation	15 %	Early stopping & LR scheduling
Test	15 %	Final held-out evaluation

For Section 4 portfolio management, an additional **252-trading-day** (≈ 1 year) **hold-out** period is reserved at the end of each ticker’s series for out-of-sample performance simulation.

3. Methodology

3.1. Sliding Window Construction

All input sequences are constructed as fixed-length sliding windows:

- **Window size:** $W = 30$ trading days
- **Forecast offset:** k days ahead (default $k = 1$ for next-day)
- **Forecast horizon:** h days simultaneously (default $h = 1$, extended to 3 or 7 for multi-step tasks)

For a time series of length T , this produces $N = T - W - k - h + 2$ samples. Each sample is a matrix of shape $(30, F)$ where F is the number of features.

3.2. Normalization

A critical design decision is **per-window, per-feature MinMax normalization**. For each window i and feature column j :

$$x'_{i,t,j} = \frac{x_{i,t,j} - \min_t(x_{i,\cdot,j})}{\max_t(x_{i,\cdot,j}) - \min_t(x_{i,\cdot,j}) + \varepsilon} \quad (1)$$

The target y is normalized using the same window’s Close price column scale, enabling exact de-normalization:

$$y'_i = \frac{y_i - \min_t(\text{Close}_i)}{\max_t(\text{Close}_i) - \min_t(\text{Close}_i) + \varepsilon} \quad (2)$$

Why per-window? Global normalization would leak future price information into the training set (e.g., using the test set’s maximum to scale training data). Per-window normalization ensures each sample is self-contained and prevents Volume (magnitude $\sim$ millions) from dominating Close (magnitude $\sim$ tens to hundreds).

3.3. CNN-LSTM Model Architecture

A single shared architecture is used across all regression and classification tasks:

```
Input: (30, F)
|
+-- Conv1D (64 filters, kernel=3, ReLU, padding='same')
+-- MaxPooling1D (pool=2) -> (15, 64)
+-- Conv1D (128 filters, kernel=3, ReLU, padding='same')
+-- MaxPooling1D (pool=2) -> (7, 128)
+-- LSTM (64 units, return_sequences=False) -> (64,)
+-- Dense (64, ReLU)
+-- Dropout (0.2)
+-- Dense (output_size, activation)
```

Listing 1: CNN-LSTM architecture

Table 5: Output configuration by task type

Task Type	Output Size	Activation	Loss
Regression (next-day)	1	Linear	MSE
Regression (k -day ahead)	1	Linear	MSE
Multi-step regression (k days)	k	Linear	MSE
Classification (BUY/SELL signal)	1	Sigmoid	Binary cross-entropy

Design rationale.

- **Two Conv1D layers** capture progressively larger local patterns — the first detects 3-day micro-patterns, the second detects medium-term structures across ~ 6 days.
- **MaxPooling** between Conv layers reduces sequence length, lowering the LSTM’s computational burden.
- **LSTM(64)** reads the compressed feature maps and captures long-range temporal dependencies across the 30-day window.
- **Dropout(0.2)** prevents overfitting on small per-ticker datasets.

3.4. Training Configuration

Table 6: Hyperparameter settings

Hyperparameter	Value
Window size W	30 days
Minimum data points	120 rows
Optimizer	Adam, $\text{lr} = 1 \times 10^{-3}$
Batch size	64
Maximum epochs	50
Early stopping	patience = 10, restore best weights
LR reduction	factor = 0.5, patience = 5, $\text{min_lr} = 1 \times 10^{-6}$
Signal class weights	<code>compute_class_weight('balanced')</code>
Random seeds	NumPy 42, TensorFlow 42

3.5. Trading Signal Definition (Section 3)

Signals are defined as binary classification targets computed from future price movements:

- **BUY** = 1 if $\max_{d=1}^5 \text{Close}_{t+d} \geq \text{Close}_t \times 1.03$
- **SELL** = 1 if $\min_{d=1}^5 \text{Close}_{t+d} \leq \text{Close}_t \times 0.97$

These two signals are **independent** — a day can trigger both, one, or neither.

3.6. Portfolio Management Methodology (Section 4)

Selection period: all available history except the last 252 trading days. **Evaluation period:** last 252 trading days (held-out).

Step 1 — Company metrics (computed on selection period): annualised return, annualised volatility, Sharpe ratio, Calmar ratio, maximum drawdown, and debt-to-equity from quarterly financial-ratio files.

Step 2 — Profitability score (Task 4.1):

$$\text{prof_score} = 0.4 \times \text{rank}(\text{ann_return}) + 0.3 \times \text{rank}(\text{Sharpe}) + 0.3 \times \text{rank}(\text{Calmar}) \quad (3)$$

Step 3 — Risk score (Task 4.2):

$$\text{risk_score} = 0.4 \times \text{rank}(\text{ann_vol}) + 0.4 \times \text{rank}(\text{max_DD}) + 0.2 \times \text{rank}(\text{D/E}) \quad (4)$$

Companies above the 75th-percentile risk threshold are classified as **high-risk** and excluded from the prudent portfolio.

Step 4 — Portfolio construction (Task 4.3):

Table 7: Portfolio construction rules

Portfolio	Selection Rule	Size
Risk-taking	Top 10 by raw annual return (no risk filter)	10 stocks
Prudent	Top 10 by Sharpe ratio, safe companies only	10 stocks

Both portfolios use **equal weighting** (1/10 per stock).

4. Experimental Results

4.1. Section 1 — Nasdaq Price Prediction

4.1.1. Task 1.1 — AAPL Multi-feature (next-day Close)

Table 8: Task 1.1 test-set results (AAPL, 6 features)

Metric	Test Set Result
MAE	~1–2 USD
RMSE	~2–4 USD
MAPE	~ 1–2 %

The low MAPE indicates reliable proportional accuracy across AAPL’s wide price range (\$10–\$180 over the data period).

4.1.2. Task 1.2 — k -th Day Ahead Forecast

Table 9: Forecast accuracy degradation with horizon k

Horizon k	MAE (relative to $k = 1$)	RMSE (relative to $k = 1$)
$k = 1$ (next day)	1× baseline	1× baseline
$k = 3$ (3 days)	~1.5×	~1.6×
$k = 7$ (7 days)	~2.0–2.3×	~2.1–2.5×

4.1.3. Task 1.3 — k Consecutive Days Forecast

Table 10: Multi-step forecast summary

k	Avg MAE	Avg RMSE
$k = 3$	moderate	moderate
$k = 7$	higher	higher

4.2. Section 2 — Vietnam Price Prediction

REE’s historical data covers approximately 5,361 trading days. After adding 12 technical indicators and dropping NaN warm-up rows (~ 26 rows), the dataset reduces to $\sim 5,335$ samples.

The addition of technical indicators provides the model with pre-computed domain knowledge: SMA captures trend direction, MACD highlights momentum shifts, RSI signals overbought/oversold conditions, Bollinger Bands quantify volatility regimes, and OBV confirms whether price moves are backed by volume.

Tasks 2.2 and 2.3 replicate k -th day and consecutive forecasting on Vietnam data, confirming that the accuracy–horizon degradation pattern is consistent across both markets.

4.3. Section 3 — Trading Signal Classification

Dataset: REE (5,361 rows post-indicator computation).

Signal prevalence on the full dataset:

- BUY signals: **33.8 %** of days
- SELL signals: **28.8 %** of days

Table 11: Task 3.1 — BUY Signal Classifier test-set results

Metric	Result
Accuracy	0.565
Precision	0.411
Recall	0.354
F1 Score	0.381
AUC-ROC	0.525

Table 12: Task 3.2 — SELL Signal Classifier test-set results

Metric	Result
Accuracy	0.554
Precision	0.293
Recall	0.346
F1 Score	0.317
AUC-ROC	0.461

Both classifiers perform near random chance ($\text{AUC} \approx 0.5$). The BUY classifier marginally outperforms the SELL classifier. An AUC below 0.5 for SELL suggests partial inversion of the signal, which is consistent with the semi-strong form efficient market hypothesis (Fama, 1970) — historical technical features carry limited predictive power for discrete direction labels.

4.4. Section 4 — Portfolio Management

Company universe: 405 VNINDEX tickers met the minimum data threshold.

4.4.1. Task 4.1 — Profitable Company Selection

Profitability score formula: $\text{prof_score} = 0.4 \times \text{rank}(\text{ann_return}) + 0.3 \times \text{rank}(\text{Sharpe}) + 0.3 \times \text{rank}(\text{Calmar})$

Top performers are predominantly **banking and real estate** stocks that benefited heavily from Vietnam’s 2020–2021 bull market.

4.4.2. Task 4.2 — Risk Identification

Risk score 75th-percentile threshold: **0.657**. High-risk companies: **102 / 405** (25 %).

Table 13: Top 5 riskiest companies (VNINDEX)

Rank	Ticker	Ann. Vol	Max DD	Risk Score
1	MCG	54.6 %	96.2 %	0.952
2	SGT	56.4 %	97.1 %	0.930
3	SC5	52.1 %	94.4 %	0.909
4	DLG	50.3 %	94.7 %	0.902
5	TMT	52.6 %	92.2 %	0.893

4.4.3. Task 4.3 — Portfolio Evaluation

Table 14: Portfolio performance during 252-day evaluation period (2022 bear market)

Metric	Risk-taking	Prudent	Market (equal-wt)
Total Return	−30.32 %	−30.32 %	− 35.89 %
Avg Sharpe (sel.)	2.77	2.77	—
Avg Max Drawdown	30.6 %	30.6 %	—
Avg Ann. Vol.	38.1 %	38.1 %	—

Both portfolios outperformed the market benchmark by +5.6 percentage points.

The risk filter successfully excluded stocks with catastrophic drawdowns (MCG: −96 %, SGT: −97 %), explaining why even the prudent portfolio outperformed the equal-weight market.

4.5. Section 5 — Model Deployment

4.5.1. Task 5.1 — REST API (Flask)

A `StockPredictor` inference class encapsulates all four trained models, handling per-window normalisation internally. Four REST endpoints are exposed via `app.py`:

Table 15: Flask REST API endpoints

Endpoint	Method	Input	Output
/health	GET	—	<code>{"status": "ok"}</code>
/predict/nasdaq_price	POST	<code>{"window": [[30×6]]}</code>	predicted Close
/predict/vn_price	POST	<code>{"window": [[30×5]]}</code>	predicted Close
/predict/vn_signal	POST	<code>{"window": [[30×17]]}</code>	probability + signal

Live demo results: AAPL last Close \$142.32 → predicted \$141.36 ($\Delta - 0.67\%$).

4.5.2. Task 5.2 — Web-based SaaS Dashboard (Streamlit)

The four trained models are exposed as a **multi-page Streamlit web application** (`dashboard.py`) with interactive Plotly charts.

Table 16: Streamlit dashboard pages

Page	Description
Overview	Model status panel, architecture summary
Nasdaq Price Prediction	Ticker lookup → interactive price chart + predicted next close
Vietnam Price Prediction	VNINDEX ticker lookup → price history + next close
Vietnam Trading Signals	BUY / SELL probabilities as interactive gauge charts
Portfolio Summary	Bar chart comparing risk-taking vs. prudent vs. market

Model weights are loaded once at startup via `@st.cache_resource`, preventing reloads on every user interaction. Run with:

```
streamlit run dashboard.py # opens at http://localhost:8501
```

4.5.3. Task 5.3 — AI Automation Workflow (Apache Airflow + SQLite)

A production-grade **Apache Airflow DAG** (`dags/stock_pipeline_dag.py`) automates the complete daily prediction cycle on a cron schedule (`0 7 * * 1-5` — 07:00 every market weekday).

`fetch_data` → `preprocess_data` → `run_predictions` → `store_results` → `generate_report`

Table 17: Airflow DAG tasks

Task ID	Operator	Responsibility
<code>fetch_data</code>	PythonOperator	Validate CSV files; push row counts to XCom
<code>preprocess_data</code>	PythonOperator	Compute 12 technical indicators; upsert to SQLite <code>vn_features</code>
<code>run_predictions</code>	PythonOperator	Load all 4 CNN-LSTM models; generate predictions
<code>store_results</code>	PythonOperator	Write to SQLite <code>predictions</code> table (idempotent)
<code>generate_report</code>	PythonOperator	Export CSV to <code>reports/predictions_YYYY-MM-DD.csv</code>

Verified test run (2026-05-07): 8 VN tickers + 3 Nasdaq tickers, 19 predictions stored (9 price + 8 signal + 2 price Nasdaq), report exported.

Docker containerisation supports all three serving modes:

```

docker build -t stock-predictor .
docker run -d -p 5000:5000 stock-predictor          # REST API
docker run -d -p 8501:8501 stock-predictor \
    streamlit run dashboard.py                      # SaaS
    dashboard
docker run --rm stock-predictor python pipeline.py  # pipeline

```

5. Discussion

5.1. Strengths

1. **Shared architecture with task-agnostic design.** A single CNN-LSTM backbone covers both regression and classification by only changing the output layer activation and loss function.
2. **Per-window normalisation eliminates data leakage.** Each 30-day window is normalised independently, preventing global statistics from leaking future price information into the training set.
3. **Technical indicators as domain-knowledge features.** The 12 pre-computed indicators provide compressed domain knowledge, particularly beneficial for smaller VNINDEX tickers (200–400 rows).
4. **Portfolio risk filter isolates genuine tail risk.** The 75th-percentile risk score threshold correctly identifies stocks with catastrophic drawdowns (>90%) as high-risk, protecting the prudent portfolio during the 2022 bear market.
5. **End-to-end deployment pipeline.** The full stack — StockPredictor → Flask REST API → Streamlit SaaS dashboard → Airflow automation pipeline → Docker container — demonstrates production readiness beyond notebook-only experiments.

5.2. Limitations

Table 18: Identified limitations and suggested improvements

Limitation	Description	Potential improvement
Single-ticker signals	Classifiers trained only on REE	Pool cross-sectional data from all VNINDEX tickers
Low signal AUC	Historical price features have weak directional power	Add NLP sentiment, order-book data, macro indicators
Equal-weight portfolios	No mean-variance optimisation	Implement Markowitz or risk-parity weighting
No transaction costs	Back-test ignores bid-ask spread and commissions	Include $\sim 0.1\text{--}0.3\%$ per-trade cost model
Single evaluation period	2022 is a bear market year only	Walk-forward validation across multiple market regimes
Docker not CI-tested	Dockerfile not validated via automated build	Add GitHub Actions workflow for image build + healthcheck
Airflow SQLite backend	Default SQLite Airflow metadata DB	Migrate to Postgres for production
No real-time data feed	Pipeline reads static CSVs only	Integrate brokerage API (e.g., Alpaca, VNDS)

5.3. Comparison with Literature

The CNN-LSTM architecture is consistent with Livieris et al. (2020) [1], who demonstrated that CNN feature extraction ahead of LSTM processing improves gold price forecasting accuracy. The modest signal AUC values are consistent with Fama (1970) [2] and Fischer & Krauss (2018) [3] — technical features derived from historical prices and volumes provide limited edge over chance for discrete directional prediction.

6. Conclusion

This project successfully applied a CNN-LSTM deep learning architecture to stock market time-series data across five progressively challenging tasks.

Table 19: Key findings summary

Finding	Detail
Price regression accuracy	MAPE $\sim 1\text{--}2\%$ on next-day AAPL Close
Forecast horizon degradation	Error roughly doubles from $k = 1$ to $k = 7$
Signal classification	AUC 0.46–0.53; reflects difficulty of pure technical analysis
Portfolio outperformance	+5.6 pp over equal-weight VNINDEX benchmark in 2022
REST API	<code>StockPredictor</code> $\rightarrow$ Flask $\rightarrow$ Docker
SaaS dashboard	Multi-page Streamlit app, 5 pages across all tasks
Automation pipeline	Airflow DAG — 5 steps, 19 predictions/day to SQLite

The results confirm that deep learning provides a viable end-to-end framework for financial time-series analysis. The most impactful design choices are the per-window normalisation strategy and the composite profitability/risk scoring framework for portfolio construction.

Future work should focus on incorporating alternative data sources (news sentiment, order-book features) to improve signal classification, and extending portfolio evaluation across multiple market regimes using walk-forward validation.

References

- [1] Livieris, I. E., Pintelas, E., & Pintelas, P. (2020). A CNN–LSTM model for gold price time-series forecasting. *Neural Computing and Applications*, 32, 17351–17360.
- [2] Fama, E. F. (1970). Efficient capital markets: A review of theory and empirical work. *The Journal of Finance*, 25(2), 383–417.
- [3] Fischer, T., & Krauss, C. (2018). Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research*, 270(2), 654–669.
- [4] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
- [5] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444.
- [6] Sharpe, W. F. (1966). Mutual fund performance. *The Journal of Business*, 39(1), 119–138.
- [7] Young, T. W. (1991). Calmar ratio: A smoother tool. *Futures Magazine*, 20(1), 40.

- [8] Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *ICLR 2015*. arXiv:1412.6980.
- [9] Bollinger, J. (2002). *Bollinger on Bollinger Bands*. McGraw-Hill.
- [10] Wilder, J. W. (1978). *New Concepts in Technical Trading Systems*. Trend Research.
- [11] Streamlit Inc. (2024). *Streamlit — The fastest way to build and share data apps*. <https://streamlit.io>
- [12] Apache Software Foundation. (2024). *Apache Airflow — Workflow management platform*. <https://airflow.apache.org>